

TP NetOps — Enchaînement des jalons

Déployer, provisionner, vérifier et observer un cœur MPLS L3VPN. Un tronc de quatre jalons (J1 → J4), plus une branche observabilité (O1 · O2 · O3) qui se greffe après J3. La branche n'est pas un rang : elle ne dépend pas de J4, et J4 ne dépend pas d'elle.

Jalon	Ce qui doit être vrai	Tâches attendues	Couche de la stack utilisée
J1 Le lab existe	La topologie imposée tourne, entièrement décrite dans un fichier.	— Fichier de topologie versionné — 13 conteneurs actifs (9 cEOS + 4 hôtes Linux) — Joignabilité hors bande des 9 nœuds Arista — Version d'EOS relevée sur un nœud — Une commande qui détruit tout, une autre qui reconstruit tout	containerlab + Docker + cEOS (+ Git)
J2 Le service tourne	Le L3VPN des deux clients fonctionne, et rien n'a été tapé à la main sur un équipement.	— Environnement Ansible : ansible.cfg, inventaire généré par containerlab (pas écrit à la main), group_vars/ + host_vars/ — Playbooks par couches : cœur (IGP + LDP) → plan de contrôle VPN (MP-BGP VPNv4) → VRF et raccordement PE-CE → configuration des CE — Playbook de checkup en escalier : un barreau par couche, any_errors_fatal, arrêt au premier barreau cassé — Le checkup couvre : sous-jacent, plan de contrôle, plan de données, trafic	Ansible via eAPI (JSON-RPC sur HTTPS/443, Ansible initie)
J3a Reproductibilité	Le lab détruit intégralement puis reconstruit revient au même état, prouvé conforme.	— destroy + deploy + rejeu complet de la chaîne, joué en direct — Checkup vert à l'arrivée, zéro geste manuel sur un équipement	Aucun outil nouveau — c'est une propriété de J1 + J2
J3b Idempotence	Rejouer la chaîne sur un lab déjà configuré ne change rien.	— Second passage complet sur lab configuré — Obtenir le signal mesurable (changed = 0) et savoir le nommer — Savoir distinguer idempotence et conformité : une ligne posée à la main hors de vos fichiers ne sera pas retirée par un rejeu	Aucun outil nouveau — propriété distincte de J3a, notée séparément
J4 La configuration est un artefact	La configuration voulue d'un équipement existe comme un texte que l'on peut regarder, indépendamment de son application.	— Gabarits Jinja2 portant la structure de la configuration — Données séparées portant ce qui varie d'un équipement à l'autre — Afficher le rendu d'un gabarit pour un équipement donné sans l'appliquer — Contrainte forte : le checkup de J2 ne doit pas être modifié — preuve par l'historique Git	Jinja2 (+ Ansible)

Jalon	Ce qui doit être vrai	Tâches attendues	Couche de la stack utilisée
O1 Branche observabilité — la collecte	Un collecteur souscrit aux 9 nœuds Arista et expose ce qu'il reçoit.	— Configuration par fichier versionné, pas par une commande jetable — Survit à la fermeture de la session : service systemd — Journalisation active — un service non diagnosticable n'est pas livrable — Rien n'est installé sur les équipements : le serveur gNMI est intégré à EOS	gNMI (dans EOS) + gnmic — modèle push
O2 Branche observabilité — le stockage	Prometheus, sur la VM Tools, collecte l'endpoint exposé par O1.	— Déploiement par Docker Compose, dans un projet séparé de tout ce qui existe déjà sur la VM — Scrape de l'endpoint du collecteur (port 9804) — Versions épinglées — latest interdit, c'est une conséquence directe de J3 — Vérifier au préalable que le port traverse bien entre les deux VM	Prometheus — modèle pull (Prometheus va chercher)
O3 Branche observabilité — la visualisation	Un tableau de bord lisible, provisionné par fichier.	— Source de données ET tableau de bord provisionnés par fichier, sans un seul clic — Afficher un débit (rate), pas un compteur brut, avec une unité juste — Courbes identifiables : nom de l'équipement et de l'interface — Deux expériences documentées : test de panne (prédire par écrit, puis mesurer) et test de chemin	Grafana (PromQL vers Prometheus)